



Credit Card Payment Verification System

Ryan Fraser

CTEC 393

Professor Courtney Lamar

May 8, 2026



Project Overview

Technologies Used

- HTML
- CSS
- PHP
- MySQL
- JavaScript
- InfinityFree

Features

- Payment submission form
- Database storage
- Query display page
- API using fetch()



Database Tables

Tables Created

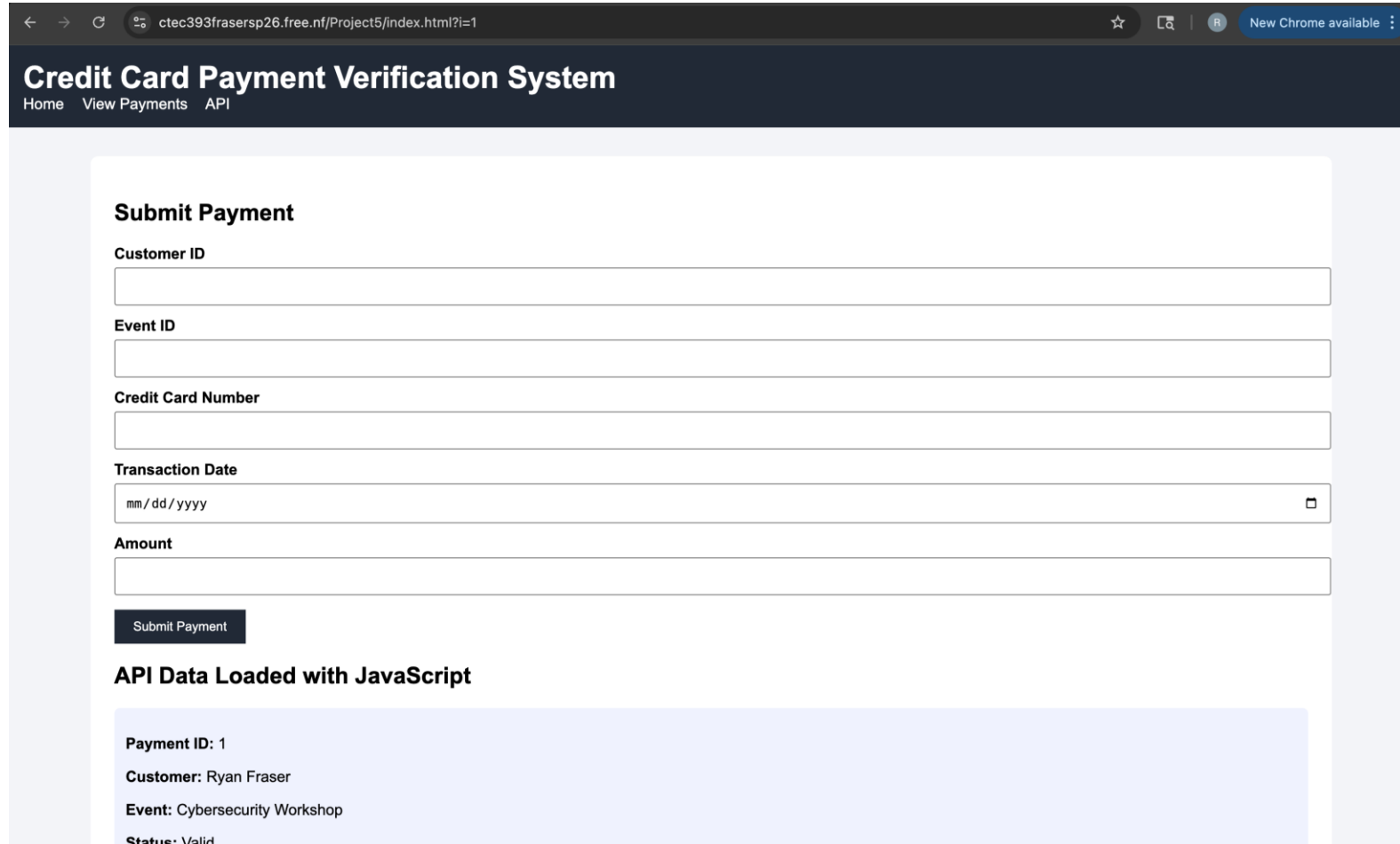
- Customer
- Event
- Payment

Relationships

- Payment links Customers and Events
 - CustomerID connects payments to customers
 - EventID connects payments to events
-

Demonstrating the Application

<https://ctec393frasersp26.free.nf/Project5/index.html>



← → ↻ 📄 ctec393frasersp26.free.nf/Project5/index.html?i=1 ☆ 📄 ⓘ New Chrome available

Credit Card Payment Verification System

Home View Payments API

Submit Payment

Customer ID

Event ID

Credit Card Number

Transaction Date

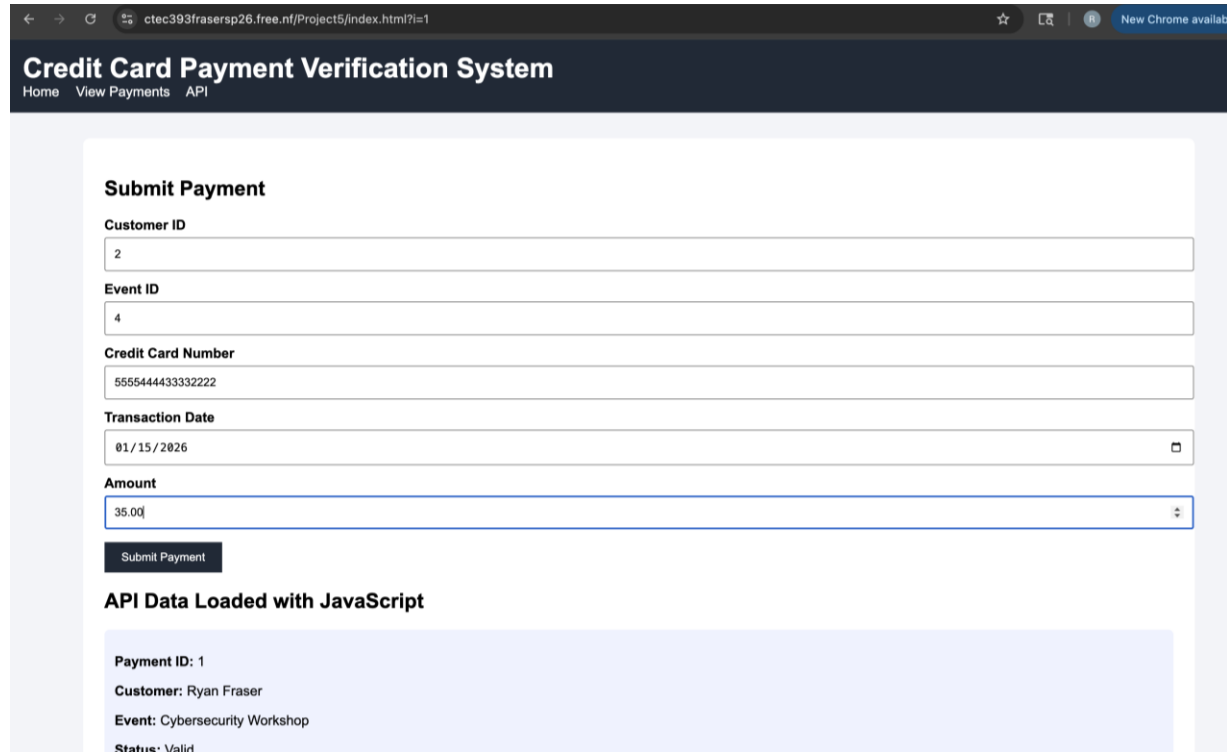
Amount

Submit Payment

API Data Loaded with JavaScript

Payment ID: 1
Customer: Ryan Fraser
Event: Cybersecurity Workshop
Status: Valid

Insertion Operation



A screenshot of a web browser showing the 'Credit Card Payment Verification System' interface. The browser's address bar displays 'ctec393frasersp26.free.nf/Project5/index.html?i=1'. The page has a dark blue header with the title and navigation links: 'Home', 'View Payments', and 'API'. The main content area is white and contains a 'Submit Payment' form. The form has five input fields: 'Customer ID' (containing '2'), 'Event ID' (containing '4'), 'Credit Card Number' (containing '5555444433332222'), 'Transaction Date' (containing '01/15/2026' with a calendar icon), and 'Amount' (containing '35.00' with a currency dropdown). A dark blue 'Submit Payment' button is at the bottom of the form. Below the form, a section titled 'API Data Loaded with JavaScript' shows payment details in a light blue box: 'Payment ID: 1', 'Customer: Ryan Fraser', 'Event: Cybersecurity Workshop', and 'Status: Valid'.

Credit Card Payment Verification System
Home View Payments API

Submit Payment

Customer ID
2

Event ID
4

Credit Card Number
5555444433332222

Transaction Date
01/15/2026

Amount
35.00

Submit Payment

API Data Loaded with JavaScript

Payment ID: 1
Customer: Ryan Fraser
Event: Cybersecurity Workshop
Status: Valid

Form Filled Out



Submission Success

<https://ctec393frasersp26.free.nf/Project5/index.html>

Showing rows 0 - 6 (7 total, Query took 0.0004 seconds.)

SELECT * FROM `payment`

Profiling

Edit inline

Edit

Explain SQL

Create PHP code

Refresh

Show all

Number of rows: 25

Filter rows: Search this table

Sort by key: None

+ Options

		PaymentID	CustomerID	EventID	CardNum	Valid	TransactionDate	Amount
☐	EditCopyDelete	1	1	1	453201511283036	Valid	2026-05-01	25.00
☐	EditCopyDelete	2	2	2	1234567890123456	Invalid	2026-05-02	15.00
☐	EditCopyDelete	3	3	3	4485275742308327	Valid	2026-05-03	20.00
☐	EditCopyDelete	4	4	4	4716108999716531	Valid	2026-07-01	35.00
☐	EditCopyDelete	5	5	5	6011000990139424	Valid	2026-07-15	30.00
☐	EditCopyDelete	6	1	5	1234567812345678	Invalid	2026-07-20	30.00
☐	EditCopyDelete	7	2	4	5555444433332222	Invalid	2026-01-15	35.00

Check all

With selected: EditCopyDeleteExport

Show all

Number of rows: 25

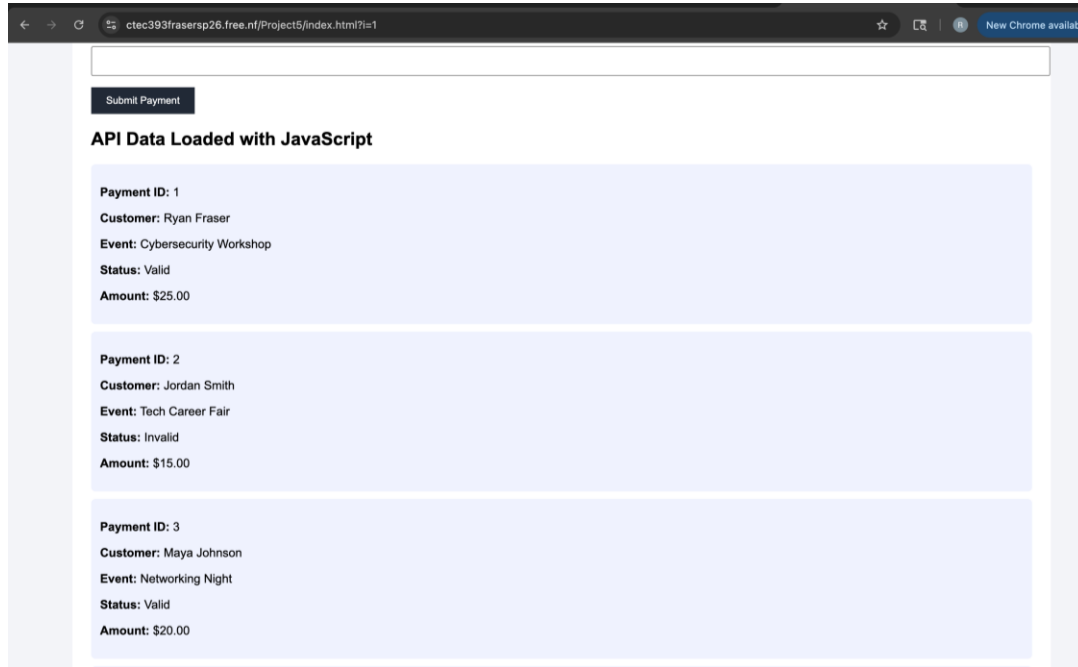
Filter rows: Search this table

Sort by key: None

Query Result

← → ↻ ctec393frasersp26.free.nf/Project5/display.php ☆ R New Chrome available ⋮ Payment Records Home View Payments API						
Payment ID	Customer	Event	Card Number	Valid	Date	Amount
1	Ryan Fraser	Cybersecurity Workshop	453201511283036	Valid	2026-05-01	\$25.00
2	Jordan Smith	Tech Career Fair	1234567890123456	Invalid	2026-05-02	\$15.00
3	Maya Johnson	Networking Night	4485275742308327	Valid	2026-05-03	\$20.00
4	Andre Carter	AI Security Conference	4716108999716531	Valid	2026-07-01	\$35.00
6	Ryan Fraser	Ethical Hacking Seminar	1234567812345678	Invalid	2026-07-20	\$30.00
7	Jordan Smith	AI Security Conference	5555444433332222	Invalid	2026-01-15	\$35.00

<https://ctec393frasersp26.free.nf/Project5/display.php>



API Demonstration

<https://ctec393frasersp26.free.nf/Project5/api.php>

Relationship / Query Explanation

Relationship Between Tables

- Tables Connected
- Customer
- Event
- Payment

How They Connect

- CustomerID links a payment to a customer
- EventID links a payment to an event

Example

- Customer Ryan Fraser registers for the Cybersecurity Workshop
- A payment record is created
- The payment stores:
 - CustomerID = 1
 - EventID = 1

The screenshot shows a database management interface for a server at sql311.infinityfree.com, database: if0_41460840_creditcardverification. The top navigation bar includes tabs for Structure, SQL, Search, Query, Export, Import, Operations, Routines, and Designer. Below this is a 'Filters' section with a search box. The main area displays a table structure overview with columns: Table, Action, Rows, Type, and Collation. It lists three tables: customer (4 rows, InnoDB, latin1_swedish_ci), event (7 rows, InnoDB, latin1_swedish_ci), and payment (7 rows, InnoDB, latin1_swedish_ci). A summary row shows 3 tables, a sum of 18 rows, and MyISAM engine with latin1_swedish_ci collation. Below the table list are controls for 'Check all' and 'With selected:'. At the bottom, there is a 'Create table' dialog with a 'Name' field and a 'Number of columns' dropdown set to 4. A 'Console' tab is visible at the very bottom.

Table	Action	Rows	Type	Collation
customer	Browse Structure Search Insert Empty Drop	4	InnoDB	latin1_swedish_ci
event	Browse Structure Search Insert Empty Drop	7	InnoDB	latin1_swedish_ci
payment	Browse Structure Search Insert Empty Drop	7	InnoDB	latin1_swedish_ci
3 tables	Sum	18	MyISAM	latin1_swedish_ci

AI Reflection

How AI Helped

- Provided sample database data for tables
- Helped explain database and PHP error messages
- Explained foreign keys and table relationships
- Helped explain how JavaScript fetch() works with APIs



What I'm Proud Of

Two Things I'm Proud Of

- 1. Successfully Connecting the Website to the Database
 - Connected InfinityFree hosting with MySQL
 - Used PHP to insert and retrieve data

Fixed connection and database errors

- 2. Creating a Working API with JavaScript fetch()
 - Built an API page that returns JSON data
 - Used JavaScript fetch() to retrieve information
 - Displayed dynamic payment data from the database